

Lecture 02 - Rendering Pipeline

This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Source chunk	course_text_parts/03_lectures/02_rendering-pipeline.txt
Extracted slide pages	94
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Slide walkthrough

Each row gives a short reading comment for one extracted slide page. Use it to turn sparse slide bullets into a human-readable path through the lecture.

Page	Slide title	How to read it
1	Untitled slide	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
2	Object-based Rendering	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
3	2.1 Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
4	2.1 Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
5	Geometry-based Rendering 1/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
6	Geometry-based Rendering 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
7	Application Stage	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
8	Geometry Stage	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
9	Rasterization Stage	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
10	2.2 OpenGL Overview	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
11	OpenGL History	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
12	OpenGL	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
13	OpenGL vs. Vulkan	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
14	OpenGL Libraries	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
15	OpenGL Functionality	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
16	OpenGL Context	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
17	OpenGL Rendering Process	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
18	Window Creation with GLFW 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
19	Window Creation with GLFW 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
20	OpenGL Naming Conventions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
21	2.3 OpenGL Rendering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
22	OpenGL Rendering (until V2.1)	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
23	OpenGL Rendering (since V3.0)	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
24	1. Vertex Specification	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
25	2. Vertex Data Transfer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
26	3. Memory Layout Description	Read this as local shading. Name the normal, light vector, view vector, material term, and where the computation happens.
27	4. VAO Drawing 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
28	4. VAO Drawing 2/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
29	5. GPU Memory Release	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
30	Rendering Initialization 1/2	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
31	Rendering Initialization 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
32	Rendering	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
33	Additional Attributes	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
34	2.4 OpenGL Rendering Pipeline	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
35	The Rendering Pipeline 1/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
36	The Rendering Pipeline 2/2	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
37	Shader Programming – History 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
38	Shader Programming – History 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
39	Assembler Shader	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
40	High-Level Shading Languages 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
41	High-Level Shading Languages 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
42	Shader Types	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
43	Vertex Shader Concepts	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
44	Vertex Shader Examples 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
45	Vertex Shader Examples 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
46	Vertex Shader Coordinate Systems 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
47	Vertex Shader Coordinate Systems 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
48	Local Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
49	World Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
50	View Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
51	Clip Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
52	Screen Space	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
53	Coordinate Systems	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.

Page	Slide title	How to read it
54	Vertex Shader Variables	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
55	Fixed Function Vertex Postprocessing	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
56	Fixed Function Rasterization	Read this as a coordinate-space step. Name the input space, the matrix or operation, and the output space.
57	Fragment Shader Concept 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
58	Fragment Shader Concept 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
59	Shader Example	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
60	GLSL Shader Functions	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
61	Shader Usage	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
62	Binding Shaders	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
63	Setting Uniforms	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
64	Shader Performance	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
65	Shader Development Environments 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
66	Shader Development Environments 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
67	2.5 Fragment Tests and Operations	Read this as continuous-to-discrete conversion. Ask which samples are covered and which attributes are interpolated.
68	Fragment Tests and Operations 1/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?

Page	Slide title	How to read it
69	Fragment Tests and Operations 2/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
70	Alpha Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
71	Stencil Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
72	Depth Test	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
73	Image Composition and Mixture	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
74	Blending Application	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
75	Blending Process 1/2	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
76	Blending Process 2/2	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
77	Applying Blending	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
78	Blending Examples	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
79	2.6 Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
80	Viewport	Read this as camera geometry. Track how view-space positions become clip, normalized, and screen coordinates.
81	Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
82	OpenGL Framebuffers 1/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
83	OpenGL Framebuffers 2/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

Page	Slide title	How to read it
84	OpenGL Framebuffers 3/3	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
85	Double Buffering	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
86	2.7 Pixel-based Rendering	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
87	Pixel-based Rendering Operations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
88	Image Operations	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
89	Write Pixels into Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
90	Read Pixels from Framebuffer	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
91	Formats & Data Types	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.
92	coordinate systems in the geometry stage	Read this as the main data-flow story: scene/model data is processed step by step until only valid framebuffer updates remain.
93	Literature and other sources used in this chapter	Read this slide by connecting the bullet terms causally: what problem is being solved, what data is used, and what output is produced?
94	Guide: The Official Guide to Learning OpenGL	Read this as pipeline state and data flow. Ask which OpenGL object or shader stage owns the data at this point.

02.1 Rendering Process

Source location: Section 2.1

Commentary

Object-based rendering starts with scene objects represented as primitives. The pipeline transforms vertices, assembles primitives, rasterizes them into fragments, and lets only some fragments become pixels. The important detail is that the representation changes at every stage.

The application stage prepares data and state. The geometry stage handles positions and primitives. The rasterization stage creates fragments and applies fragment operations. This separation helps you explain both theory questions and black-screen OpenGL bugs.

Mental model

Vertices become primitives, primitives become fragments, and fragments compete to become pixel updates.

Check yourself

What is the difference between a fragment and a final pixel?

02.2 OpenGL Overview

Source location: Section 2.2

Commentary

OpenGL is an API for controlling a graphics pipeline. It is not a renderer by itself in the sense of a single command that understands your scene. You create a context, provide buffers, set state, compile shaders, bind objects, and issue draw calls.

Modern OpenGL is stateful and shader-based. Many errors happen because the programmer assumes that a later call carries more information than it really does. The GPU uses the currently bound objects and current state at draw time.

Mental model

OpenGL draw calls consume the state machine as it exists right now.

Check yourself

Why can binding the wrong vertex array or shader program produce a valid draw call with wrong output?

02.3 OpenGL Rendering

Source location: Section 2.3

Commentary

Older OpenGL exposed a fixed-function style where many transformations and lighting choices were configured through predefined calls. Modern OpenGL expects you to write shader programs and explicitly manage data flow.

This is not just historical trivia. It explains why shader code, vertex attributes, uniforms, and buffer objects are central in the exercises. If the shader expects an attribute and the vertex layout does not provide it, the pipeline has no magic fallback.

Mental model

Modern OpenGL makes the data path explicit; that gives control but also creates responsibility.

Check yourself

Which data usually goes into vertex attributes, and which data usually goes into uniforms?

02.4 OpenGL Rendering Pipeline

Source location: Section 2.4

Commentary

The vertex shader runs per vertex and usually outputs clip-space position plus attributes for later interpolation. Primitive assembly groups vertices into points, lines, or triangles. Clipping handles geometry outside the view volume. Rasterization creates fragments.

The fragment shader computes candidate fragment output, often using interpolated attributes, uniforms, and textures. After that, tests and operations such as depth testing, stencil testing, blending, and masking decide what reaches the framebuffer.

Mental model

Programmable shaders compute values; fixed pipeline stages decide coverage, interpolation, tests, and output rules.

Check yourself

If your geometry appears in wireframe but colors are wrong, which stages become suspicious?

02.5 Fragment Tests, Framebuffer, and Pixel-Based Rendering

Source location: Sections 2.5-2.7

Commentary

Fragment tests are the gatekeepers after shading. A fragment can have a perfectly valid color and still fail because depth, stencil, scissor, masking, or blending state prevents a visible update. This is why final pixels are a subset of generated fragments.

The framebuffer stores the final render targets: color buffers, depth buffers, stencil buffers, or custom attachments. Pixel-based rendering starts from image data instead of geometric primitives, which is useful for image processing but conceptually different from the object-to-pixel pipeline.

Mental model

The framebuffer is the destination; fragment operations are the rules for writing into it.

Check yourself

Why can two fragments with different colors produce only one visible pixel color?

End-of-lecture summary

If you remember only one thing from Lecture 02, remember this: This lecture turns the vague idea of rendering into a concrete sequence. It is the most important debugging map in the course: if an image is wrong, the pipeline tells you where the error could have entered.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.